

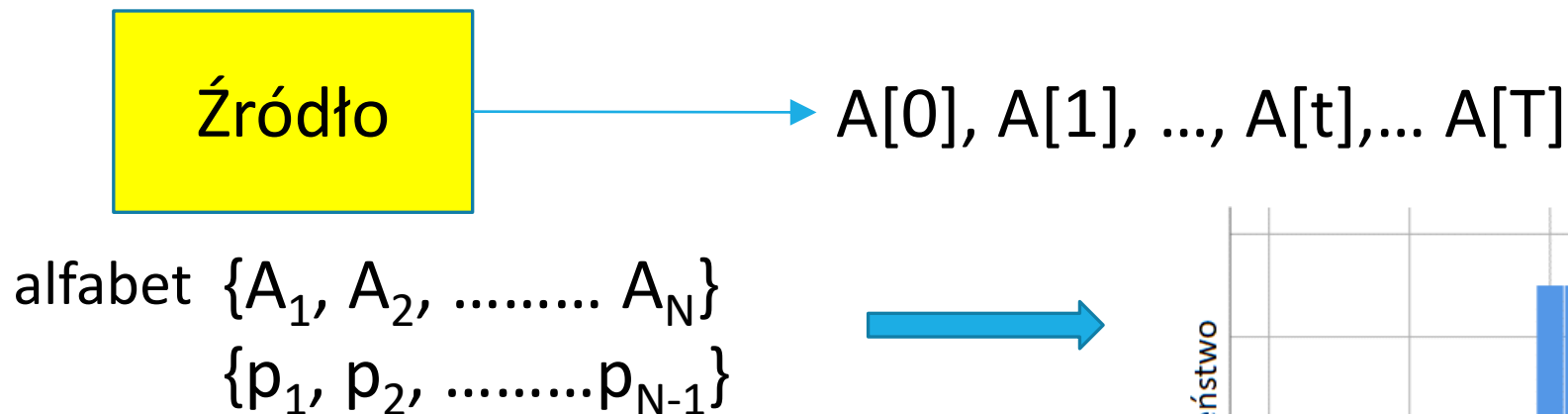
Podstawy teorii informacji

WSTĘP DO MULTIMEDIÓW

Źródło danych

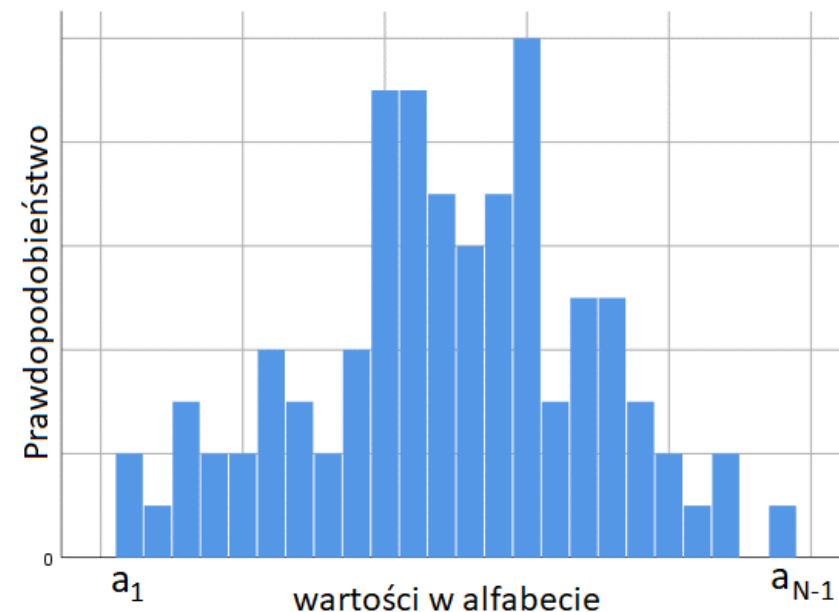
Źródło danych generuje zdarzenia (symbole) zgodnie z pewnym rozkładem prawdopodobieństwa

- proces stochastyczny
- zbiór wszystkich możliwych wartości symboli nazywany jest alfabetem



Modele **dyskretne** – zakłada się, że źródło jest

- **stacjonarne**: prawdopodobieństwo jest niezmiennie względem przesunięć w czasie
- **ergodyczne**: każda generowana sekwencja ma te same właściwości statystyczne



Informacja

Każde zdarzenie niesie ze sobą pewną informację – $i(A)$

- większe prawdopodobieństwo zdarzenia – mniejsza ilość informacji
- mniejsze prawdopodobieństwo zdarzenia – większa ilość informacji

Miara informacji to informacja własna zdarzenia:

$$i(A) = \log_b \frac{1}{p(A)} = -\log_b p(A)$$

Informacja uzyskana w wyniku wystąpienia dwóch niezależnych zdarzeń jest sumą informacji uzyskanych w wyniku wystąpienia każdego z tych zdarzeń osobno:

$$i(AB) = -\log_b [p(A) \cdot p(B)] = \log_b \frac{1}{p(A)} + \log_b \frac{1}{p(B)} = i(A) + i(B)$$

Różne podstawy logarytmu → inne jednostki wartościowania:

- 2 → bit – wartościowanie binarne
- 256 → bajt – wartościowanie bajtowe
- 10 → hartley
- e → nat

Konwersja za pomocą
mnożenia przez stałą

Entropia

Średnia informacja symboli źródła ważona prawdopodobieństwami:

$$H = \sum_n p(A_n) i(A_n) = - \sum_n p(A_n) \log_b p(A_n)$$

- Entropia wyznacza **granice** kompresji bezstratnej
- Większa entropia to większa średnia informacja
 - Małe prawdopodobieństwo -> duża entropia
 - Duże prawdopodobieństwo -> mała entropia
- Uwaga: źródło stanowią wszystkie elementy ścieżki przetwarzania uczestniczące w formowaniu symboli
 - Każdy element może zmieniać rozkłady prawdopodobieństw
 - Obraz + predykcja pikseli ma inny rozkład niż sam obraz -> inna entropia

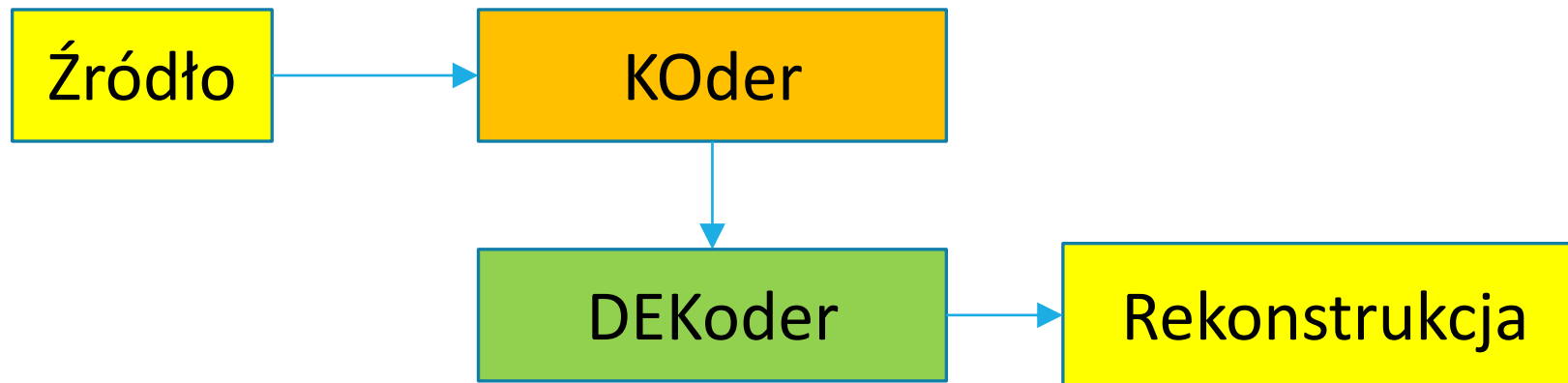
System kodowania

KOD – reguła (algorytm) tworzenia reprezentacji kodowej (efektywnego ciągu bitowego) danych źródłowych

KODEK (KOder + DEKoder) – realizacja algorytmu kodowania (oprogramowanie, sprzęt)

KODOWANIE – proces tworzenia reprezentacji kodowej (koderem według ustalonego kodu)

DEKODOWANIE – proces odtwarzania reprezentacji źródłowej na podstawie reprezentacji kodowej



Koder binarny

Koder binarny przetwarza sekwencję symboli wejściowych generowanych przez źródło na sekwencję bitów



Koder z książką kodów

- przypisanie każdemu symbolowi określonej sekwencji bitowej (słowa kodowego)
- kodowanie sekwencji symboli polega na wysłaniu do wyjścia sekwencji odpowiednich słów kodowych – sekwencja bitowa budowana jest przyrostowo
- z punktu widzenia realizowalności istotne są tylko takie kody, które są jednoznacznie dekodowalne

Algorytm kodowania przyrostowego

1. Odczytaj z wejścia kolejny symbol do zakodowania – σ
2. Jeśli wejście jest puste, to zakończ i zamknij strumień bitowy
3. Emituj na wyjście (dopisz do strumienia bitowego) słowo kodowe symbolu σ
4. Kontynuuj od kroku 1

Kody przedrostkowe

- Szczególną grupą kodów jednoznacznie dekodowalnych są kody przedrostkowe
- Przedrostkowość kodu oznacza, że żadne słowo kodowe nie jest początkiem (przedłużeniem) innego słowa kodowego
- Każdy kod przedrostkowy jest jednoznacznie dekodowalny
- Nie każdy kod jednoznacznie dekodowalny jest kodem przedrostkowym

kod przedrostkowy

	Symbol	Słowo kodowe
1	A	0
2	B	10
3	C	110
4	D	111

kod jednoznacznie dekodowalny
nie-przedrostkowy

	Symbol	Słowo kodowe
1	A	0
2	B	01
3	C	11

Dekodowanie 011 11 11 11 11

Warunek sumy dwójkowej (1)

Jeżeli kod jest jednoznacznie dekodowalny, to długości słów kodowych $l_1 \dots l_N$ spełniają warunek sumy dwójkowej (nierówność Krafta-McMillana):

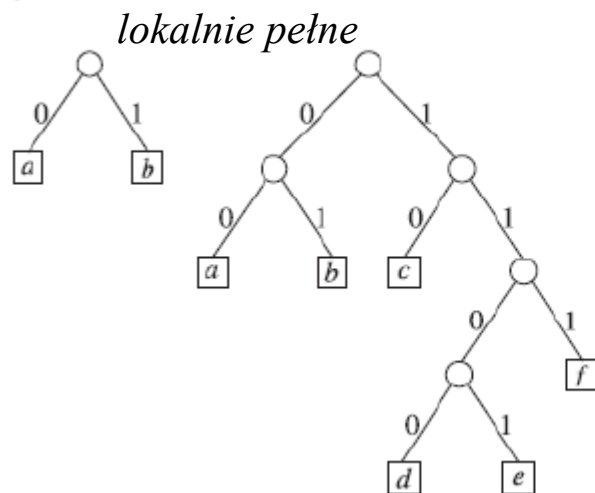
$$2^{-l_1} + 2^{-l_2} + \dots + 2^{-l_N} \leq 1 \quad \longleftrightarrow \quad \sum_{i=1}^N 2^{-l_i} \leq 1$$

gdzie N jest liczbą słów kodowych (rozmiarem alfabetu)

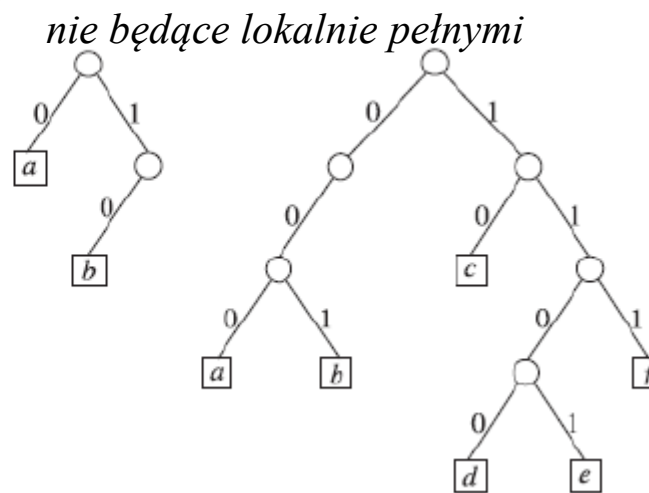
Jeżeli liczby naturalne $l_1 \dots l_N$ spełniają warunek sumy dwójkowej, to istnieje kod przedrostkowy o N słowach kodowych o długościach $l_1 \dots l_N$

Z powyższych własności wynika, że każdy kod jednoznacznie dekodowalny można „przekształcić” – z zachowaniem długości słów kodowych – w kod przedrostkowy

Warunek sumy dwójkowej (2)



$$\sum_{i=1}^N 2^{-l_i} = 1$$



$$\sum_{i=1}^N 2^{-l_i} < 1$$

- Lokalna pełność – każdy węzeł ma dwa potomki zwiększające długość o jeden bit
- Każdy bit długości przepoławia wagę sumowanego czynnika – dwie połowy sumują się do wagi węzła
- Ze względu na proste dekodowanie, w praktyce wykorzystuje się kody przedrostkowe

Dekoder przedrostkowy

Zastosowanie kodu przedrostkowego pozwala na prostą realizację dekodera

Algorytm dekodera przedrostkowego:

1. Zainicjuj α jako pustą sekwencję bitową
2. Jeśli wejście jest puste, to zakończ
3. Odczytaj kolejny bit z wejścia i dopisz go do α
4. Jeśli α jest słowem kodowym dla pewnego symbolu σ , to emituj na wyjście symbol σ i kontynuuj od kroku 1
5. Kontynuuj od kroku 3

Kody o stałej długości

Najprostszym kodem binarnym jest kod o stałej długości

- słowa kodowe są kolejnymi liczbami zapisanymi w naturalnym kodzie dwójkowym
- wszystkie słowa kodowe mają taką samą długość, wynikającą z liczby symboli w alfabecie
- przypisanie słów kodowych do symboli zależy od ich pozycji w alfabecie

	Symbol	Słowo kodowe
1	A	000
2	B	001
3	C	010
4	D	011
5	E	100

ABCADAEABCA



000 001 010 000 011 000 100 000 001 010 000
(33 bity)

Kody o zmiennej długości

Zmiana długości poszczególnych słów kodowych może przyczynić się do skrócenia strumienia bitowego

	Symbol	Słowo kodowe
1	A	00
2	B	01
3	C	10
4	D	110
5	E	111

ABCADAEABCA



00 01 10 00 110 00 111 00 01 10 00
(24 bity)

Optymalny kod przedrostkowy

- Różne kody dadzą w wyniku sekwencje bitowe o różnych długościach
- Najlepszym kodem będzie ten, który zakoduje sekwencję symboli z użyciem jak najmniejszej liczby bitów
- W celu znalezienia kodu optymalnego należy uwzględnić rozkład prawdopodobieństwa symboli
 - symbole występujące często – krótsze słowa kodowe
 - symbole występujące rzadko – dłuższe słowa kodowe
- Miarą optymalności kodu jest średnia długość słowa kodowego:

$$\bar{l} = \frac{l}{n} = \frac{l_1 n_1 + l_2 n_2 + \dots + l_N n_N}{n_1 + n_2 + \dots + n_N} = p_1 l_1 + p_2 l_2 + \dots + p_N l_N$$

gdzie: l – długość sekwencji bitowej,
 n – liczba zakodowanych symboli,
 l_i – długość i -tego słowa kodowego,
 n_i – liczba wystąpień i -tego symbolu,
 p_i – prawdopodobieństwo i -tego symbolu

Własności optymalnego kodu przedrostkowego

- dwa symbole o najmniejszym prawdopodobieństwie mają najdłuższe słowa kodowe – c i d , o takiej samej długości,
- dla każdego słowa kodowego k istnieje słowo kodowe k' równe k za wyjątkiem ostatniego bitu
 - można tak ustawić słowa kodowe, że $c' = d$ (czyli c i d będą różniły się tylko ostatnim bitem)
- jeśli połączymy dwa symbole o najmniejszych prawdopodobieństwach w jeden supersymbol, któremu przypiszemy słowo kodowe c bez ostatniego bitu, to tak otrzymany kod jest optymalny dla nowego alfabetu o mniejszej liczbie symboli
- suma dwójkowa w optymalnym kodzie przedrostkowym jest równa jeden

Algorytm Huffmana

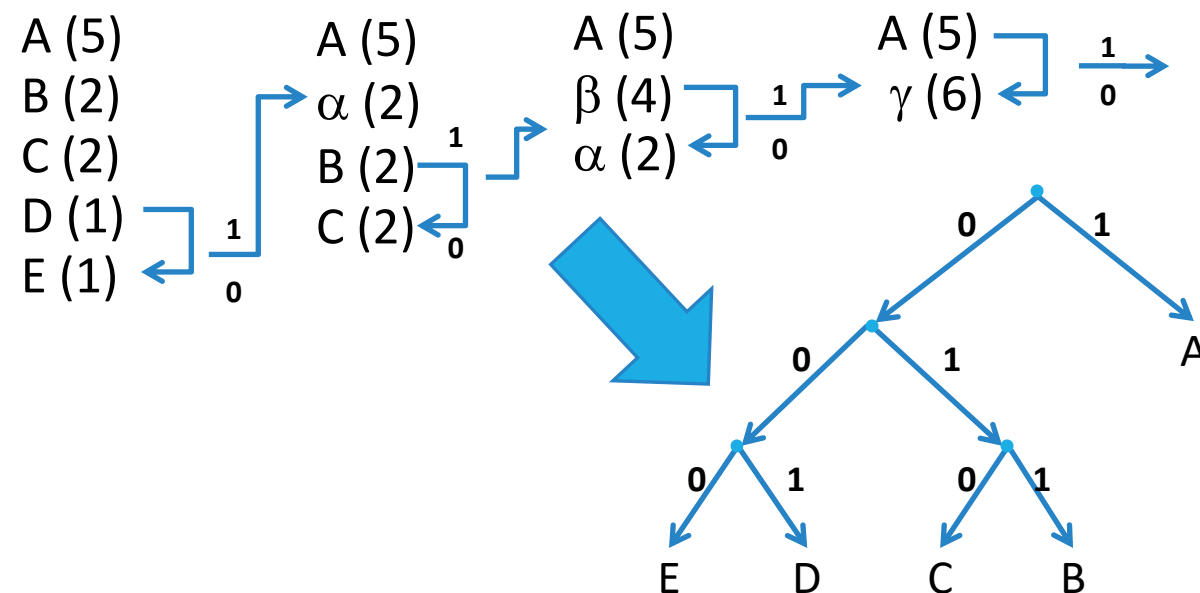
Znajdź dwa symbole o najmniejszych prawdopodobieństwach, X i Y , i zastąp je nowym symbolem Z

- słowa kodowe dla symboli X i Y zostaną utworzone ze słowa kodowego symbolu Z poprzez dodanie bitu '0' i '1'

W celu znalezienia słowa kodowego dla symbolu Z , zastąp w alfabecie symbole X i Y symbolem Z

- prawdopodobieństwo występowania symbolu Z jest sumą prawdopodobieństw symboli X i Y

Powtarzaj procedurę aż do uzyskania tylko jednego symbolu



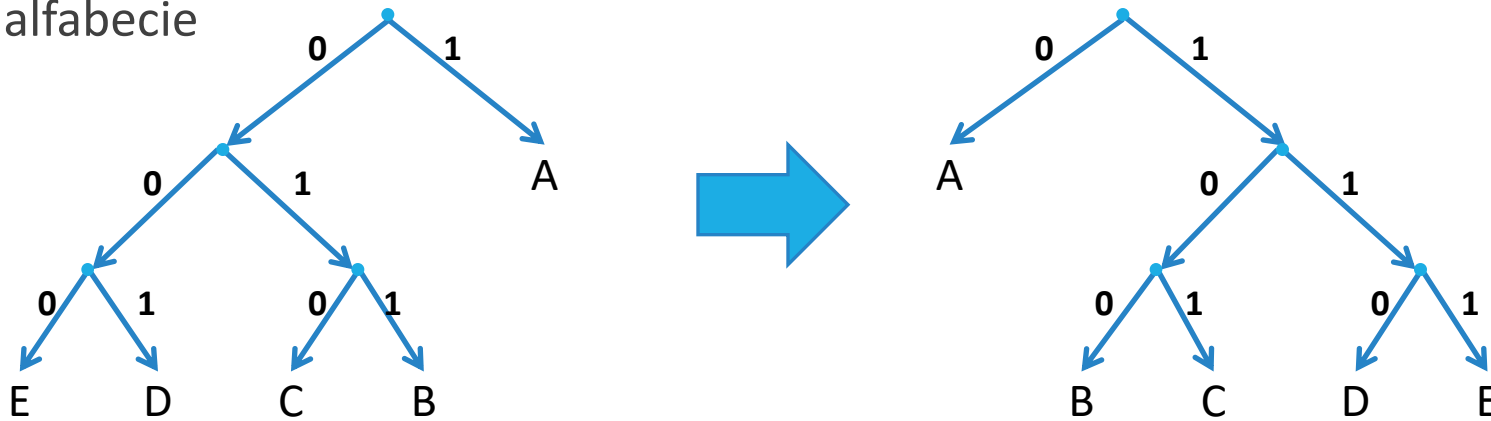
	Symbol	Słowo kodowe
1	A	1
2	B	011
3	C	010
4	D	001
5	E	000

ABCADAEABCA

1 011 010 1 001 1 000 1 011 010 1
(23 bity)

Reprezentacja znormalizowana kodu

Normalizacja kodu polega na takim przeorganizowaniu słów kodowych, aby w drzewie kodu liście uszeregowane od lewej do prawej miały głębokości niemalejące, a symbole na danym poziomie zachowywały kolejność jak w alfabecie



	Symbol	Długość kodu	Słowo kodowe
1	A	1	0
2	B	3	100
3	C	3	101
4	D	3	110
5	E	3	111

Normalizacja kodu nie ma wpływu na jego **optymalność**

- zmianie ulegają tylko wartości słów kodowych, ale nie ich długości

Znając długości słów kodowych można dokonać normalizacji kodu bez korzystania z drzewa

Algorytm budowania znormalizowanego kodu przedrostkowego o zadanych długościach słów kodowych

- uporządkuj symbole według: a) długości słów kodowych, b) pozycji w alfabecie
- najkrótsze słowo kodowe składa się z samych zer
- kolejne słowa kodowe tworzymy z poprzedniego przez inkrementację wartości (arytmetyczne dodanie wartości jeden) oraz dopisanie sekwencji zer aż do uzyskania pożądanej długości

Kod przedrostkowy a entropia

Średnia długość kodu przedrostkowego nie może być mniejsza niż entropia rozkładu prawdopodobieństwa symboli

$$\bar{l} = \sum_{i=1}^N p_i l_i \geq H = \sum_{i=1}^N p_i \log_2 \frac{1}{p_i}$$

Średnia długość optymalnego kodu przedrostkowego różni się od entropii mniej niż o jeden

$$\bar{l} < H + 1$$

Przykład: Niech $l_i = \left\lceil \log_2 \frac{1}{p_i} \right\rceil = \log_2 \frac{1}{p_i} + \varepsilon, \quad 0 \leq \varepsilon < 1$

Kod o długościach słów kodowych równych l_i spełnia warunek sumy dwójkowej. Można zatem zbudować odpowiedni kod przedrostkowy

$$2^{-l_i} = 2^{\log_2 p_i - \varepsilon_i} = p_i \cdot 2^{-\varepsilon_i} \leq p_i \Rightarrow \sum_i 2^{-l_i} \leq \sum_i p_i = 1$$

Dla powyższego kodu

$$\bar{l} = \sum_i p_i l_i = \sum_i p_i \left(\log_2 \frac{1}{p_i} + \varepsilon_i \right) < \sum_i p_i \left(\log_2 \frac{1}{p_i} + 1 \right) = \sum_i p_i \log_2 \frac{1}{p_i} + \sum_i p_i = H + 1$$

Optymalny kod przedrostkowy nie może być gorszy od kodu o arbitralnie wybranych długościach słów kodowych

Grupowanie symboli alfabetu

Łącząc kolejne kodowane symbole w grupy po M symboli, tworzymy nowe symbole złożone (supersymbole), składające się na alfabet blokowy rzędu M

- prawdopodobieństwo symbolu złożonego jest iloczynem prawdopodobieństw symboli go tworzących (przy założeniu niezależności symboli)

Entropia alfabetu blokowego rzędu M jest M -krotnie większa od entropii alfabetu oryginalnego

Średnia entropia dla alfabetu blokowego

$$H_M = M \cdot H$$

Średnia długość kodu na symbol oryginalny

$$H_M \leq \bar{l}_M < H_M + 1$$

$$\bar{l}_M = \frac{l_M}{M} \Rightarrow H \leq \bar{l} < H + \frac{1}{M}$$

Grupowanie symboli przyczynia się do zwiększenia efektywności kodowania, ale zwiększa jego złożoność

Specjalne kody przedrostkowe

Kody przedrostkowe często stosowane w praktyce.

Definicja kodów dla określonych **rozkładów prawdopodobieństwa** symboli daje:

- możliwość określenia optymalnego kodu przedrostkowego **dla pewnych modeli** statystycznych bez konieczności stosowania algorytmów uniwersalnych
- uniknięcie konieczności **przekazywania** książki kodów do dekodera poprzez zbudowanie książki kodów dla reprezentatywnego zbioru uczącego

Definicja kodów dla **nieskończonych** alfabetów: Przyjmuje się, że symbole są utożsamiane z **liczbami naturalnymi**, a alfabety określone są przez przedziały liczb naturalnych, bo:

- symbole alfabetu zawsze da się uszeregować według pewnego, arbitralnie przyjętego, porządku
- zbiór liczb całkowitych można w sposób jednoznaczny przekształcić w zbiór liczb naturalnych

Kod unarny

Kod unarny $U(n)$ dla $n \geq 0$ zdefiniowany jest następująco:

- $U(n) = 1^n0 = 1...10$ (n jedynek zakończonych zerem)
- definicja rekurencyjna:
 $U(0) = 0, U(n+1)=1U(n)$
- stosowana jest również wersja, w której role zera i jedynki są zamienione:
 $U'(n) = 0^n1 = 0...01$ (n zer zakończonych jedynką)
- kod unarny jest **optymalnym** kodem przedrostkowym (kodem Huffmana) dla rozkładu prawdopodobieństwa symboli określonego ciągiem geometrycznym o pierwszym wyrazie $\frac{1}{2}$ i ilorazie $\frac{1}{2}$:

$$p_n = \frac{1}{2}^{n+1}, n = 0, 1, 2, 3, \dots$$

Kod prawie stałej długości

Kod prawie stałej długości $B_b(n)$, $0 \leq n < b$:

- ma długość $k = \lfloor \log_2 b \rfloor$ dla $0 \leq n < 2^{k+1}-b$
kolejne kody przypisywane są rosnąco, począwszy od $B_b(0) = 0\dots 0$ (k zer) – jest to binarny zapis liczby n na k bitach
- ma długość $(k+1)$ dla $2^{k+1}-b \leq n < b$
kolejne kody przypisywane są malejąco, począwszy od $B_b(b-1) = 1\dots 1$ ($k+1$ jedynek) – jest to binarny zapis liczby $2^{k+1}-b+n$ na $(k+1)$ bitach
- dla $b = 2^k$ kod prawie stałej długości jest naturalnym kodem dwójkowym (**NKB**) stałej długości równej k
- kod prawie stałej długości jest optymalnym kodem przedrostkowym dla rozkładu prawdopodobieństwa symboli **zrównoważonego** według kryterium $2min > max$ i jest identyczny ze znormalizowanym kodem Huffmana dla tego rozkładu
- rozkład zrównoważony według kryterium $2min > max$:
$$p_0 \geq p_1 \geq \dots \geq p_{b-2} \geq p_{b-1} \implies p_{b-2} + p_{b-1} \geq p_0$$

Przykład: $b = 5$, $k = \lfloor \log_2 5 \rfloor = 2$

- przedział kodowania na $k = 2$ bitach:
 $[0, 2^{k+1}-b) = [0, 3) = [0, 2]$
- przedział kodowania na $k = 3$ bitach:
 $[2^{k+1}-b, b) = [3, 5) = [3, 4]$

- słowa kodowe:

$$B_5(0) = 00$$

$$B_5(1) = 01$$

$$B_5(2) = 10$$

$$B_5(4) = 111$$

$$B_5(3) = 110$$

Kod Golomba

Kod Golomba G_b , $b > 0$, przypisuje symbolowi $n \geq 0$ sekwencję bitów $U(q)B_b(r)$ składającą się z:

- kodu unarnego $U(q)$ dla $q = \lfloor n/b \rfloor$ po którym następuje kod prawie stałej długości $B_b(r)$ dla $r = n \bmod b$
- q można interpretować jako numer przedziału, zaś r jako pozycję w danym przedziale (odległość od początku przedziału); b określa długość przedziału
- dla $b = 2^k$ kodowanie prawie stałej długości upraszcza się do naturalnego kodu dwójkowego o stałej długości równej k
- kod Golomba jest optymalnym kodem przedrostkowym dla **geometrycznego rozkładu prawdopodobieństwa** symboli z prawdopodobieństwem sukcesu p i porażki $(1-p)$: $p_n = p(1-p)^n$, parametr b (dł. przedziału) dobiera się z warunku:

$$(1-p)^b + (1-p)^{b+1} \leq 1 < (1-p)^b + (1-p)^{b-1},$$
$$b(p) = \lceil \log(2-p)/-\log(1-p) \rceil$$

- dla $p = (1-p) = 1/2$: $p_n = 1/2^{n+1}$, $b = 1$ i kod Golomba staje się równy kodowi unarnemu (przy założeniu, że $B_1(0) =$ ciąg pusty)

Kod Golomba

Przykład: $G_6(9) = ?$; $n = 9$, $b = 6$,

kod unarny:

$$q = \lfloor 9/6 \rfloor = 1, U(1) = 10$$

kod prawie stałej długości:

$$k = \lfloor \log_2 6 \rfloor = 2$$

$$r = 9 \bmod 6 = 3 \geq 2^3 - 6$$

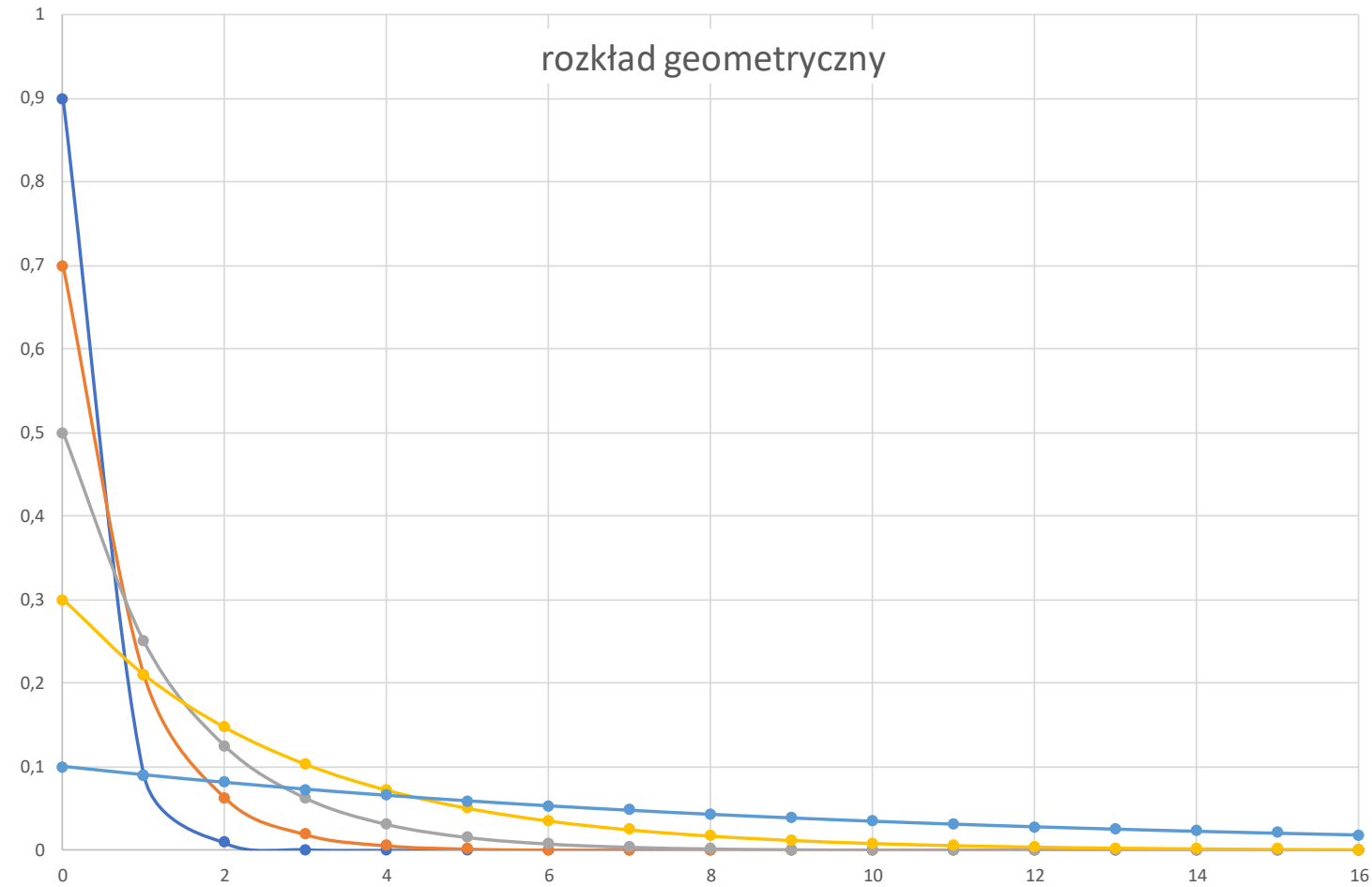
(r należy zakodować na $k+1 = 3$ bitach)

$$B_6(3) = (2^3 - 6 + 3)_2 = (5)_2 = 101$$

kod Golomba: $G_6(9) = U(1)B_6(3) = 10 \ 101$

Kolejne słowa kodowe kodu G_6 :

0: 0 00	6: 10 00	12: 110 00
1: 0 01	7: 10 01	13: 110 01
2: 0 100	8: 10 100	14: 110 100
3: 0 101	9: 10 101	15: 110 101
4: 0 110	10: 10 110	16: 110 110
5: 0 111	11: 10 111	17: 110 111



Wykładniczy kod Golomba

Długości kolejnych przedziałów zmieniają się w postępie geometrycznym:

$$b_q = 2^q,$$

- ponieważ długości przedziałów są potęgami dwójki to używany w tym kodowaniu kod B_{bq} jest naturalnym kodem dwójkowym na q bitach
- kolejne przedziały wykładniczego kodu Golomba:
[0, 0], [1, 2], [3, 6], [7, 14], [15, 30], [31, 62], [63, 126], [127, 254]...
- kolejne słowa kodowe wykładniczego kodu Golomba

0: 1	1: 01 0	3: 001 00	7: 0001 000
	2: 01 1	4: 001 01	8: 0001 001
		5: 001 10	9: 0001 010
		6: 001 11	10: 0001 011
			11: 0001 100
			12: 0001 101
			13: 0001 110
			14: 0001 111

- zastosowanie kodu U' (zera są wiodące) pozwala na proste dekodowanie – po odczytaniu odpowiedniej liczby bitów należy od uzyskanej wartości odjąć 1 aby uzyskać wartość kodowaną

Modelowanie w kodowaniu

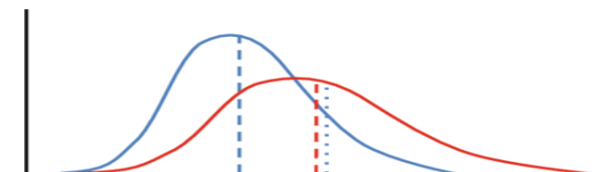
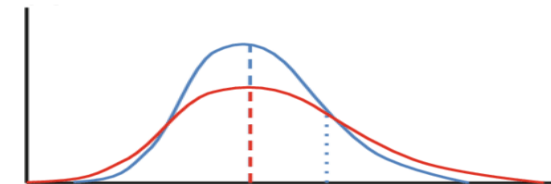
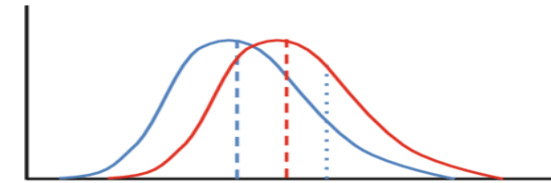
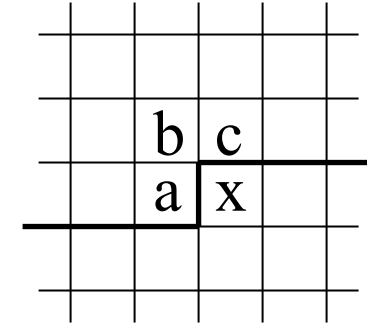
- Modelowanie prawdopodobieństw często prowadzi do lepszych współczynników kompresji przy wyższych kosztach obliczeniowych

Kontekst:

- Kodowanie kontekstowe wykorzystuje dostępne/zdekodowane dane ze sąsiedztwa do przewidywania następnej wartości i/lub prawdopodobieństwa
 - Kontekst może rozróżniać stosowane rozkłady prawdopodobieństwa i kody
 - Kontekst to np. otaczające piksele w obrazie lub poprzednie litery w zdaniu/wyrazie
- Kodowanie bezkontekstowe eliminuje zależności strukturalne w danych.

Adaptacja:

- Kodowanie statyczne zakłada, że raz wyznaczone prawdopodobieństwa nie zmieniają się
- Kodowanie adaptacyjne zakłada dostosowanie do zmieniających się prawdopodobieństw
 - Adaptacja wprzód: wysłanie kodów po wyznaczeniu modelu na jakiejś porcji danych przed ich zakodowaniem/zdekodowaniem
 - Adaptacja wstecz: prawdopodobieństwa/kody są modyfikowane po każdym zakodowanym/zdekodowanym symbolu

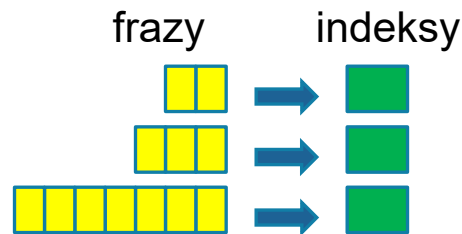


Kodowanie słownikowe

WSTĘP DO MULTIMEDIÓW

Idea kodowania słownikowego

- Koder słownikowy przypisuje pewnej sekwencji symboli ze strumienia wejściowego adres (przesunięcie, indeks) komórki słownika, z której wychodząc można tę sekwencję wygenerować
- Słownik jest z reguły dynamicznie modyfikowany, w jednakowy sposób przez koder i dekodek
- Dopasowywanie kolejnych symboli z wejścia polega na znalezieniu takiej porcji danych, która już znajduje się w słowniku
- Symbol w ciągu wejściowym występujący po dopasowanej porcji danych – δ , nazywany jest ogranicznikiem



Poszczególne metody kodowania słownikowego różnią się sposobem budowy (modyfikacji) słownika oraz sposobem traktowania ogranicznika δ :

- czy δ jest oddzielnie kodowane;
- czy δ bierze udział w aktualizacji słownika;
- czy δ jest zwracana do strumienia wejściowego (czy δ jest pierwszym symbolem kolejnej porcji symboli);

Algorytm LZ77

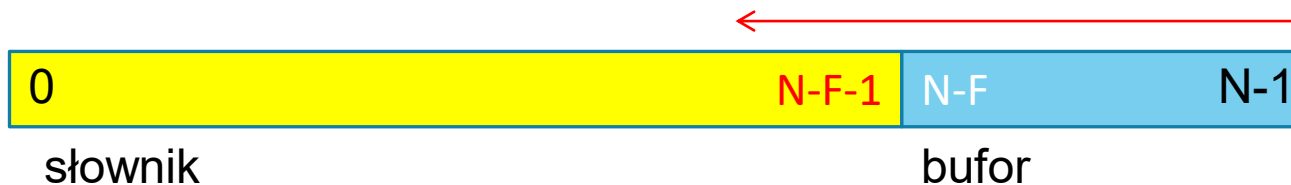
Sekwencja wejściowa przepływa przez bufor o długości N : $buf[0...(N-1)]$

Pierwsze $N-F$ pozycji bufora: $prev[0...(N-F-1)]$ obejmuje symbole już zakodowane

Ostatnie F pozycji bufora: $ahead[0...(F-1)] = buf[(N-F)...(N-1)]$ obejmuje symbole do zakodowania

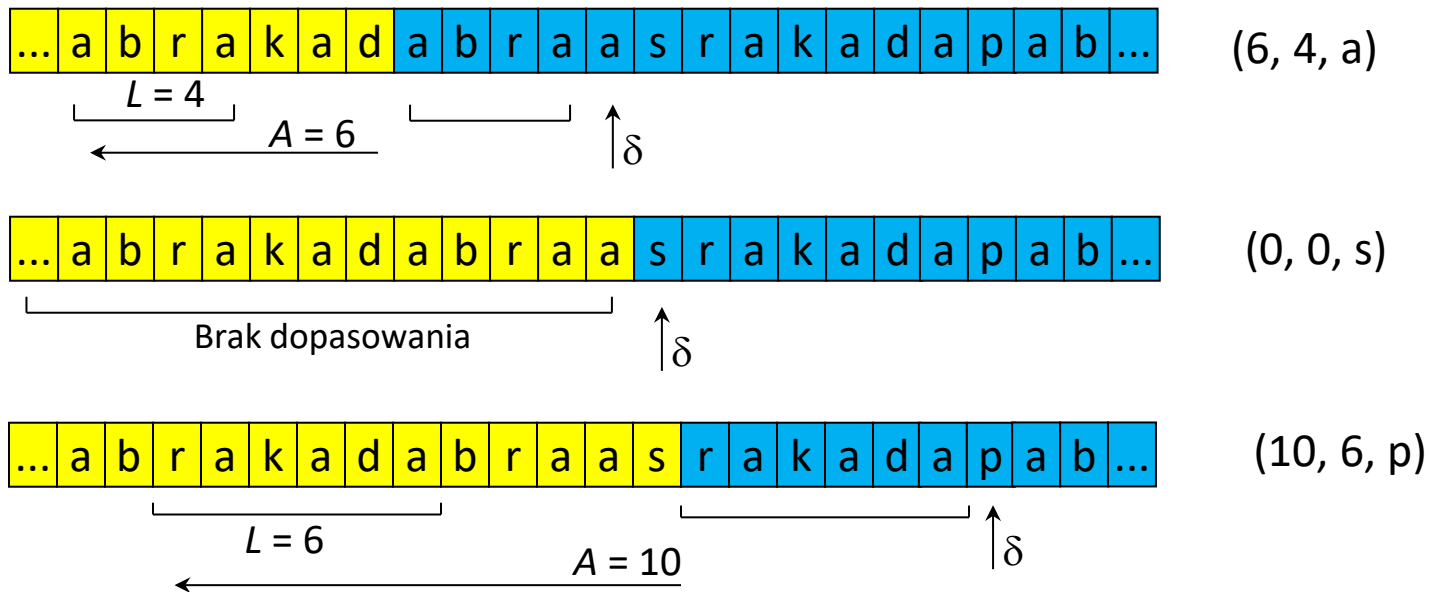
Elementami słownika są wszystkie słowa zapisane w buforze buf , rozpoczynające się w części **żółtej**; początkowa zawartość części **żółtej** jest nieistotna (koder i dekodery powinny ją inicjalizować jednakowo)

Szukamy takiego słowa (ciągu symboli) $ahead[0...(i-1)]$, które znajduje się już w $buforze[(N-F-j)...(N-F-j+i-1)]$ dla pewnego j , $0 < j \leq N-F$ (symbol $ahead[i]$ pełni rolę ogranicznika δ)



Algorytm LZ77

- Indeksem komórki słownika, pozwalającą na jednoznaczną identyfikację, jest para (A, L) ; A określa przesunięcie komórki słownika względem początku bufora *ahead*, L określa długość słowa
- Indeks (A, L, δ) wysyłane są do strumienia wyjściowego
 - kodowane przy pomocy kodu binarnego stałej długości, długość kodu może być inna dla A , L oraz δ
- Aktualizacja słownika polega na przesunięciu zawartości bufora o $L+1$ pozycji, w ten sposób δ przechodzi do części *prev*; na ostatnie $L+1$ pozycji bufora *ahead* wpływają kolejne symbole z wejścia
- Brak dopasowania sygnalizowany $L=0$



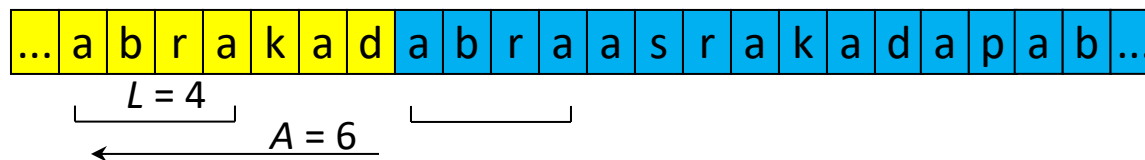
Algorytm LZSS

Modyfikacje algorytmu LZ77

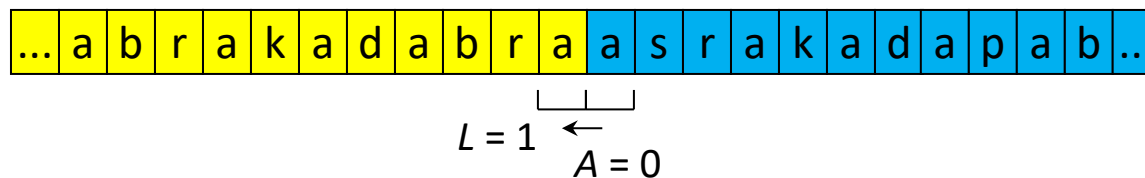
- kodowanie (A, L, δ) z użyciem kodów o zmiennej długości
- kodowanie albo pary (A, L) albo symbolu δ – algorytm LZSS
 - Rozróżnienie obu przypadków przy pomocy dodatkowego **wiodącego bitu**
 - Oszczędność bitów gdy brak dopasowania

(0, δ)

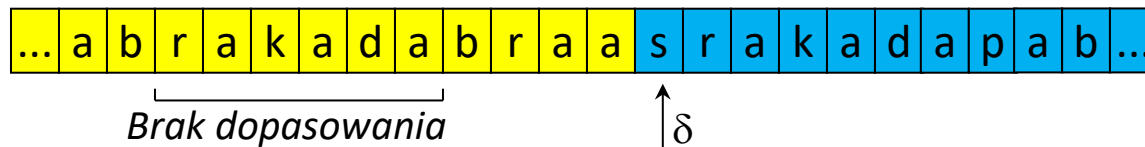
(1, A, L)



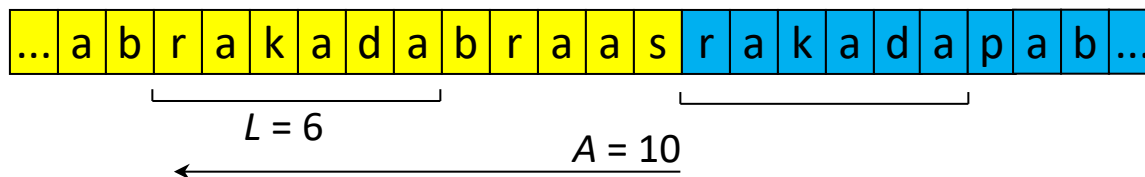
(1, 6, 4)



(1, 0, 1) lub (0, a)



(0, s)

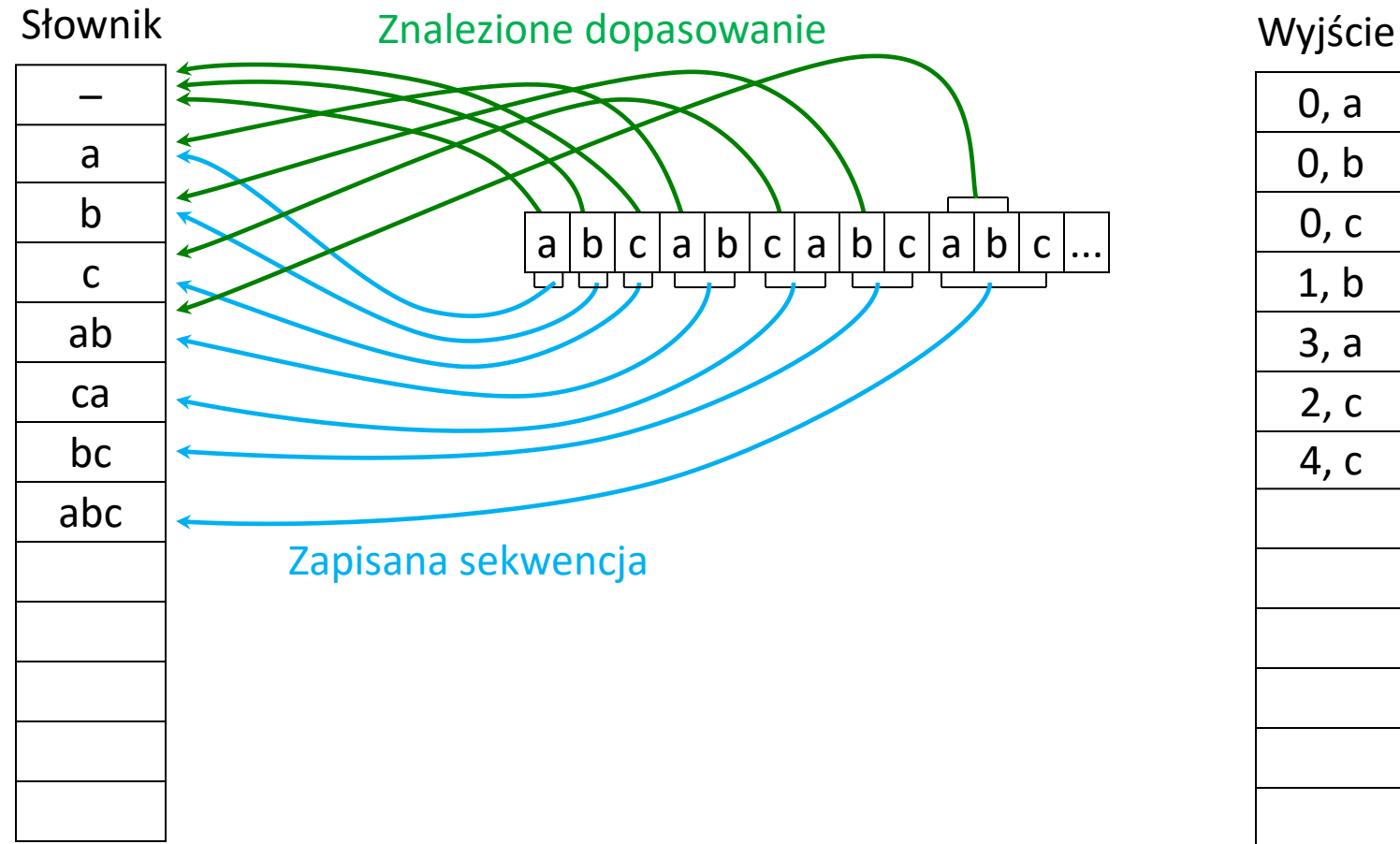


(1, 10, 6)

Algorytm LZ78

- Słownik jako rozbudowywana lista
- Początkowo słownik jest pusty
- W sekwencji wejściowej wyszukiwane jest najdłuższe słowo t , występujące już w słowniku
- Indeks słownika oraz ogranicznik δ wysyłane są do strumienia wyjściowego
 - indeksy słownika kodowane są przy pomocy kodu binarnego o stałej długości (jednakże długość kodu wzrasta wraz ze wzrostem rozmiaru słownika)
 - symbole (ograniczniki δ) kodowane są przy pomocy kodu binarnego o zmiennej długości
- Aktualizacja słownika polega na dopisaniu słowa $t\delta$.
- W przypadku wyczerpania pamięci na słownik
 - kontynuacja kodowania z ustalonym słownikiem
 - przebudowa słownika (wyrzucenie pewnych słów)
 - wyzerowanie słownika

Algorytm LZ78



Algorytm LZW

- Modyfikacja LZ78
- Początkowo słownik zawiera wszystkie słowa jednoliterowe
- W sekwencji wejściowej wyszukiwane jest najdłuższe słowo t , występujące już w słowniku
- Do strumienia wyjściowego wysyłany jest tylko indeks słownika
 - ogranicznik δ jest pierwszą literą kolejnego słowa
- Aktualizacja słownika polega na dopisaniu słowa $t\delta$

Algorytm LZW

